

How Claude Code works

Module 1 · the agent loop and what it costs

Lab: L01 First session and memory

OBJECTIVES

What you will be able to do

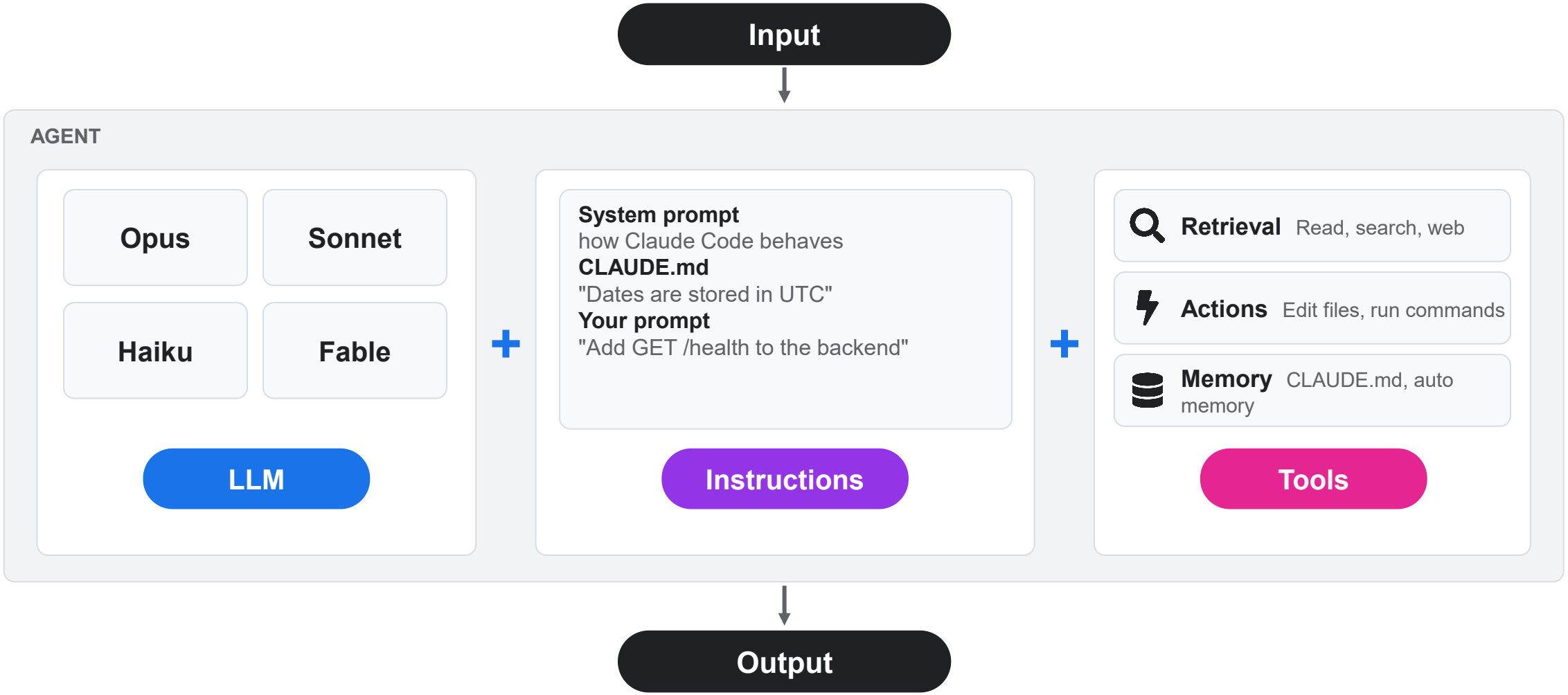
- Name the three parts of an agent and where each lives in Claude Code
- Describe what happens between your prompt and Claude's answer
- Name the five kinds of tools Claude Code uses
- Choose a surface for a job: terminal, IDE, Desktop, web, Slack or headless
- Run work in the background and share a result as an artifact
- Pick a model and effort level, and read your usage
- Say what agents are bad at, so you plan around it

From chatbot to agent

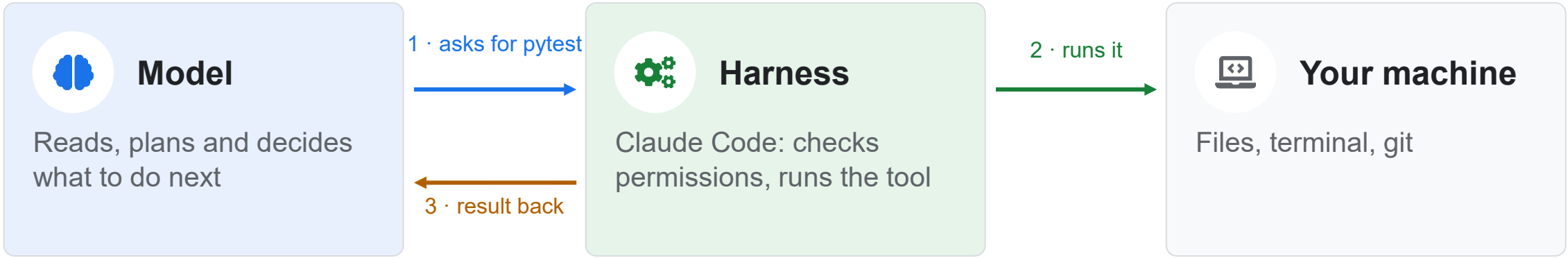
	CHATBOT	CLAUDE CODE
Input	Text you paste	Your whole project, terminal and git state
Output	Text	Edits, commands, test runs
Checks its own work	No	Yes, it runs things and reads the result
Who does the typing	You	The agent

In M00 you prompted a model that can only talk. From here on, the model can act on what you tell it.

Anatomy of an agent



Model and harness



"Claude decides"

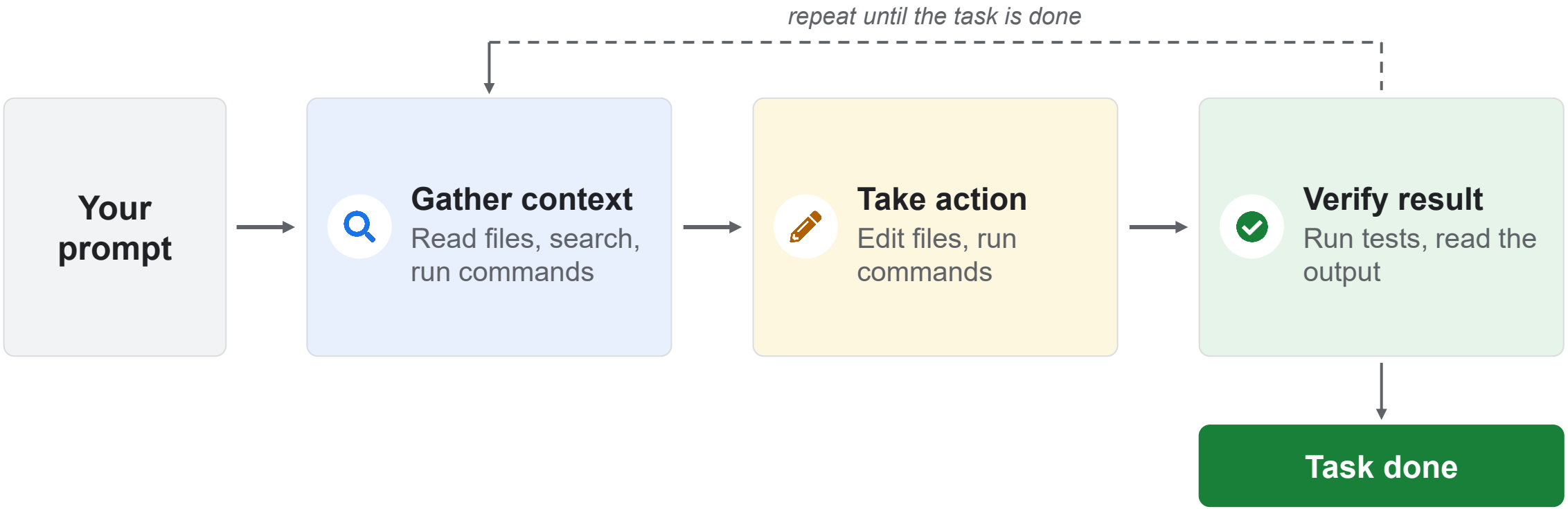
means the model. It never touches your files directly; it only asks.

"Claude Code blocks"

means the harness. Permissions and hooks live here (M8, M9).

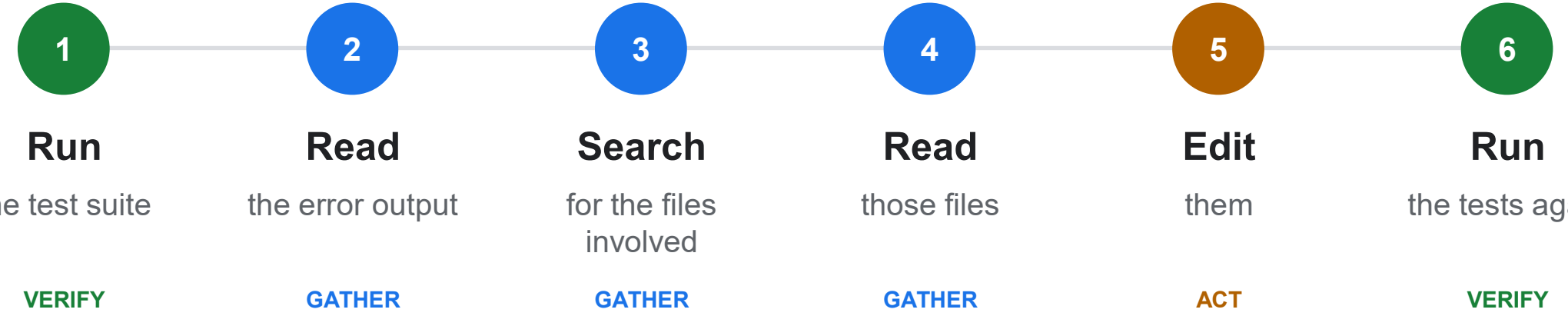
Chef and kitchen: the chef decides the dish, the kitchen has the tools and the safety rules.

The loop



You are part of the loop. Press Esc to stop immediately, or type a correction and press Enter while it works.

Example: "fix the failing tests"



Each tool result feeds the next decision. A question may need only the first phase; a bug fix cycles through all three several times.

Tools

Without tools a model can only talk. Tools are what make it act.

File operations

Read, edit, create, rename files

Search

Find files by pattern, search content with regex

Execution

Run shell commands, start servers, run tests, use git

Web

Search the web, fetch documentation

Code intelligence

Type errors, go-to-definition, find references (needs a code-intelligence plugin)

Also tools to start subagents and ask you questions. Add more with skills, hooks, MCP servers and subagents (M6 to M12).

What Claude Code can see

When you run claude in a folder

- Your files in that folder and below (files elsewhere need your permission)
- Your terminal: anything you could run
- Your git state: branch, uncommitted changes, recent commits
- Your instruction files: CLAUDE.md, or an AGENTS.md
- Auto memory: learnings Claude saved in earlier sessions
- Extensions you configured: skills, subagents, MCP servers

vs

It does not see

- Your other projects
- Your browser (unless you connect Claude in Chrome)
- Earlier sessions' conversations

`Bring a folder in with /add-dir`

Sessions are independent

Starts empty

Every new session starts with an empty context window

Saved on your machine

Under ~/.claude/projects/ so you can resume it

Tied to the folder

Switch git branches and the files change, but the conversation stays

CONTINUE, NAME OR FORK A SESSION

```
claude --continue                    # continue the latest session
claude --resume                      # pick a session to resume
/rename health-endpoint              # name it so you can find it later
claude --continue --fork-session     # fork (or /branch); original unchanged
```

What carries over between sessions is only what is written down.

1.6 · CONTEXT

The context window in Claude Code

M00: the context window is the model's desk. Here is what Claude Code puts on it.

EVERYTHING THE MODEL READS ON A TURN (ILLUSTRATIVE PROPORTIONS)



It fills as you work

Every file read and command output lands here

Old tool output cleared first

When it nears the limit

Then summarized (compaction)

Early detail can be lost

`/context`

see what is using space

`/compact`

summarize now, with a focus

`/clear`

fresh start for a new task

Persistent rules belong in files, not in chat (M3).

1.7 · SURFACES

Where it runs

SURFACE	USE IT FOR
Terminal (CLI)	Full feature set. The reference for this course
IDE extensions	VS Code and JetBrains: chat beside your code, inline diffs, @ mentions
Desktop app	Claude Code in a graphical window, no terminal needed
Web (claude.ai/code)	Run on Anthropic-managed machines, work on repos you don't have locally
Slack	Mention Claude in a thread to hand it a coding task
Headless (claude -p)	Scripts and CI, no person at the keyboard (M19)

The loop, tools and capabilities are the same everywhere. What changes is where code runs and how you interact.

Execution environments

Local

Your machine. The default. Full access to your files, tools and environment.

Cloud

Anthropic-managed VMs, or self-hosted environments your organisation runs.

Remote Control

Runs on your machine; you drive it from a browser or your phone.

Two questions pick the environment

Where must the code run? · How do I want to interact with the agent?

Agents view: sessions in the background

Research preview: the interface and shortcuts may change.

One screen lists every background session on your machine.

COMMAND	WHAT IT DOES
`claude agents`	Open agent view (all projects)
`claude agents --cwd ~/code/todo-app`	Only one project
`claude --bg "scaffold the frontend"`	Start a background session from the shell
`/bg`	Move the current conversation to the background
`/fork`	Copy it into a new background session; the original keeps going

Session states, peek and attach

STATE	MEANING
Working	Running tools or generating
Needs input	Waiting on you: a question, a permission, a prompt
Idle / Completed / Failed / Stopped	Ready, finished, errored, or stopped by you



`Ctrl+T` pin (keeps it running) `Ctrl+X` stop (twice to delete)

Background sessions in practice

Own git worktree

Sessions started with --bg or from agent view move into a worktree under .claude/worktrees/ before editing (M5)

Independent usage

Each session draws on your usage on its own, so five at once uses it five times as fast

Idle sessions

Finished and unattached for about an hour: the process stops. It resumes when you attach

The workspace must be trusted first (M8). Turn the feature off with disableAgentView.

SHELL MANAGEMENT

```
claude logs <id>        claude stop <id>
claude rm <id>          claude agents --json
```

1.9 · ARTIFACTS

Artifacts: results as shareable pages

An artifact is a live web page that Claude publishes from your session to a private URL on claude.ai. It updates in place as the session continues.

USE ONE WHEN TERMINAL TEXT IS THE WRONG MEDIUM

- **Annotated PR walkthrough**
- **A dashboard**
- **Options side by side**
- **A progress board**

EXAMPLE PROMPT

```
Make an artifact that walks through this PR with the diff annotated inline.
```

Sharing, updating and limits

Visibility	Private to you until you share. Pro and Max: a public link is the only way to share. Team and Enterprise: named people or the whole organisation; public links only if an Owner allows
Editors	Team and Enterprise: switch a person from viewer to editor; they update it through their own Claude session
Comments	Team and Enterprise: viewers comment on the page, and Claude can read and reply
Update	Ask Claude to revise it; each publish is a version. Give Claude the URL to update from another session
Find again	/artifacts lists yours and shared ones; Ctrl+] reopens the latest
Limits	One static page, no backend, strict content policy, 16 MiB
Live data	A page can call claude.ai MCP connectors each time it is viewed, using the viewer's own connection (M10)

What artifacts require

A plan

Pro, Max, Team or Enterprise. On by default for Team; an Owner enables it on Enterprise

Signed in with /login

An API key, gateway token or cloud credential cannot publish

Anthropic API provider

Not Bedrock, Google Cloud or Microsoft Foundry

Organisation policy

Not with Zero Data Retention, customer-managed keys (CMEK) or HIPAA

Off by default in claude -p, the Agent SDK and the GitHub Action. Organisation controls are in M20. Turn it off yourself in /config or with "enableArtifact": false.

Models

Opus

The default. Strong reasoning: architecture, multi-step problems

Sonnet

Daily coding. Costs less than Opus

Haiku

Simple, fast tasks; good for small subagents

Fable

The hardest, longest tasks. Never the default: you pick it

- Switch during a session with /model, or start with claude --model sonnet
- opusplan: Opus while planning, Sonnet while building
- Each model has its own prompt cache: the first request after a switch re-reads the whole conversation without cache

Opus is the default, but Sonnet handles most coding tasks well and costs less.

What the models can do

All four can do the same things. They differ in depth, speed and price.

MODEL	BEST FOR · SPEED · $ PER 1M TOKENS IN / OUT
Fable	Hardest, longest tasks · slowest · $10 / $50
Opus	The default; architecture, long runs · moderate · $4 / $20
Sonnet	Daily coding · fast · $2 / $10
Haiku	Small tasks, subagents · fastest · from $0.10 / $0.50

CHECK AND PIN

```
/model          # picker
/model opus     # alias
/model claude-opus-5-5
/status         # in use
```

Same for all four

1M-token context, 128K output, text and image input, knowledge up to June 2026.

Effort in Claude Code

M00: reasoning is a setting. In Claude Code that setting is effort.

LEVEL	USE FOR
low	Renames, formatting, quick lookups
medium	Everyday coding. Default on the 5.5 models
high	Hard bugs, multi-file changes
xhigh	Long, deep explorations
max	The hardest problems; most tokens

COMMANDS

```
/effort           # slider
/effort high      # set it
/effort auto      # back to
                  # default
```

Thinking stays on

On Opus, Sonnet and Haiku 5.5 and Fable you can't turn thinking off. Lower the effort instead.

Tokens and cost

Charged by tokens

Every turn re-sends the conversation, so long sessions cost more per message

Prompt caching

Makes the repeated part cheap

A break longer than the cache lifetime

The next message reprocesses everything

about $13

per developer per active day

$150 to $250

per developer per month

under $30

per active day for 90% of users

Figures from Anthropic for enterprise deployments. Treat these as a baseline, not a promise.

Reading your usage

COMMAND	SHOWS
`/usage`	Session tokens and estimated cost, plan limits, and what is using them
`/context`	What is filling the context window right now
`/insights`	An HTML report on how you work, from recent sessions on this machine

INSIDE /USAGE ON A PLAN

Attribution

Share used by skills, subagents, plugins, MCP servers

Recommendations

e.g. "Trim CLAUDE.md · 49k tokens per request"

Press f to fix

Claude applies the recommendations it can

```
Deciphering… (10m 27s · ↓ 5.5k tokens)
```

Spinner: output this turn has produced so far. Cost figures are estimates at list price, not an invoice.

Habits that lower cost

1

Clear often

/rename, then /clear between unrelated tasks. Stale context is paid for on every message.

2

Be specific

"Add GET /health to backend/app/main.py with a pytest test" beats "set up the backend".

From M00

3

Match model to job

Sonnet for routine work; Opus or Fable when you need it; lower the effort for simple tasks.

4

Give a way to verify

A failing test or expected output lets Claude check itself.

From M00

5

Stop early

Esc the moment it heads the wrong way; don't pay for a wrong path.

Where each habit is taught: planning in M4, instruction files and compaction in M3, delegating bulky work in M7.

1.14 · LIMITS

What agents are bad at

LIMIT FROM M00	WHAT IT LOOKS LIKE IN AN AGENT	WHERE YOU FIX IT
Context rot	Forgetting: a rule you gave 2 hours ago is ignored	**A file (M3)**
Hallucination	Guessing: fills a missing business rule with something plausible	**The repo (M3, M14)**
"Tests pass" ≠ correct	False confidence: "all tests pass" without running them	**Verify, review (M18)**
New with agents	Over-reach: edits more than you asked	**Permissions, plan first (M8, M4)**
New with agents	Remote side effects: a deploy or DB change can't be rewound	**Permissions (M8)**

Instruction files are context, not enforcement. Claude usually follows them but nothing forces it. Anything that must always happen needs a deterministic control (hooks, M9).

Your turn: read a transcript

A developer types: **`fix the failing health test`**. Claude then does the steps below. Mark each step as gather, act or verify.

1. Runs `grep -rn "health" backend/`
2. Reads `backend/app/main.py`
3. Edits `backend/app/main.py`
4. Runs `uv run pytest -q`
5. Reads the failing test output

REFERENCE

A turn on the to-do backend

```
> add a version field to the /health response and update its test

● Search("def health" in backend/app/)          <- gather
● Read(backend/app/main.py)                     <- gather
● Edit(backend/app/main.py)                     <- act
● Bash(uv run pytest tests/test_health.py -q)   <- verify
  1 failed: test_health_response
● Read(tests/test_health.py)                    <- gather (failure)
● Edit(tests/test_health.py)                    <- act
● Bash(uv run pytest tests/test_health.py -q)   <- verify
  2 passed
```

Every line is tagged gather, act or verify: the loop in a real turn.

REFERENCE

Handy commands

FROM THE SHELL	
`claude`	Start a session in this folder
`claude "explain this repo"`	Start with a first prompt
`claude -c`	Continue the latest session
`claude -r`	Pick a session to resume
`claude --model sonnet`	Start on a specific model
`claude agents`	Open agent view
`claude --version`	Installed version

IN A SESSION	
`/usage`	Cost, plan limits, what uses them
`/context`	What fills the context window
`/model`	Switch model
`/effort`	Set the effort level
`/rename`	Name this session
`/clear`	Fresh conversation
`/artifacts`	List your artifacts
`Esc`	Stop the current step
`Esc Esc`	Rewind menu (M4)

CHECK YOUR UNDERSTANDING

Question 1

Name the three parts of an agent, and where each one lives in Claude Code.

Think first, then we discuss.

CHECK YOUR UNDERSTANDING

Question 2

Which part of Claude Code decides what to do next, and which part runs the tool?

Think first, then we discuss.

CHECK YOUR UNDERSTANDING

Question 3

You start a new session on Monday. Does Claude remember Friday's conversation?

Think first, then we discuss.

CHECK YOUR UNDERSTANDING

Question 4

A long session starts ignoring a rule you gave in chat. Name two reasons it can happen and one fix.

Think first, then we discuss.

CHECK YOUR UNDERSTANDING

Question 5

Your usage is high on an API plan. Give two likely causes.

Think first, then we discuss.

CHECK YOUR UNDERSTANDING

Question 6

True or false: if CLAUDE.md says "never edit docs/PRD.md", Claude Code can never edit it.

Think first, then we discuss.